Overflow: Additional Explanation

Always keep in mind that we have to be aware of the range of values that our representations can hold. This is a straightforward calculation of the possible permutations: 2^n, where n is the number of bits in the representation.

In two's complement arithmetic, the operands must be the same length. The left-most bit is always interpreted as the sign bit. To shorten a 2's complement representation, you can remove as many bits as you like from the left, as long as they are all the same and you leave a sign bit in the left-most bit position.

Examples: suppose you have number 1111011000 and you need to shorten it to 8 bits
11011000 → **Good** (negative number that represents the same original value)
01011000 → **Bad** (positive number that represents a different value)

By the same token, you can also propagate a sign bit so that the representation occupies the correct number of bits. To do this, simply copy the most significant bit (*i.e.*, the left-most bit) as many times as needed to expand the number.

Examples: suppose you have number 1011000 and you need to expand it to 8 bits
11011000 → **Good** (negative number that represents the same original value)
01011000 → **Bad** (positive number that represents a different value)

With these ideas in mind, we can easily tell if overflow has occurred when adding two's complement values. There are two cases:

- If we are adding two numbers with ***different*** **signs**:
  - There will always be enough bits to hold the result, since the result is closer to 0 than at least one of the operands.
  - **There is <u>never</u> overflow in this case, even if there is a carry out from the last column.**
- If we are adding two numbers with the ***same*** **sign:**
  - **Overflow has occurred if the <u>sign bit of the result is different from that of the operands</u>.**
  - **This is true even if there is NO carry out from the last column**.

Ignore any carry out from the last addition, and focus instead on the most significant (left-most) bits of operands and result.

Examples:

a.		0101 0111	b.		1010 1001	c.		1100 1011	d.		1100 1011
	+	<u>0010 1010</u>		+	<u>1101 0110</u>		+	<u>1101 0110</u>		+	<u>0101 0110</u>
		1000 0001		~~1~~	0111 1111		~~1~~	1010 0001		~~1~~	0010 0001
		Overflow			**Overflow**			**NO overflow**			**NO overflow**

Reference

Brookshear, J.G. (2000). Computer Science: An Overview, 6th ed. Reading, MA: Addison-Wesley, pp. 55-59.